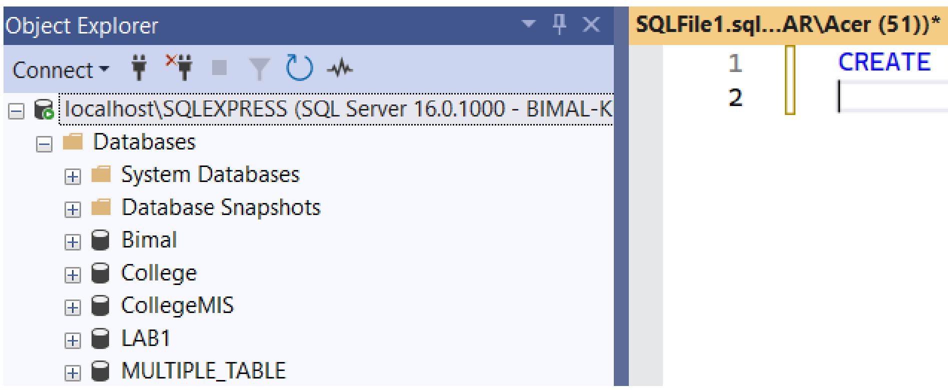
Lab Assignment 2: Use of Multiple tables

A. Creating Database:

Query: CREATE DATABASE CollegeMIS;



B. Creating Table of Student:

Query: CREATE TABLE Student (

StudentID INT PRIMARY KE Y,

Name VARCHAR(50),

Gender CHAR(1),

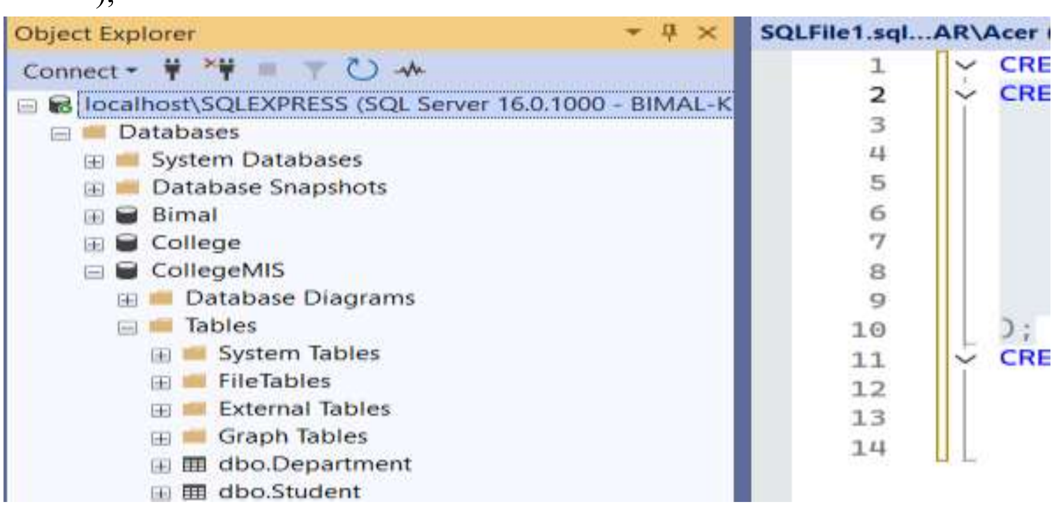
DOB DATE,

DepartmentID INT,

Email VARCHAR(100),

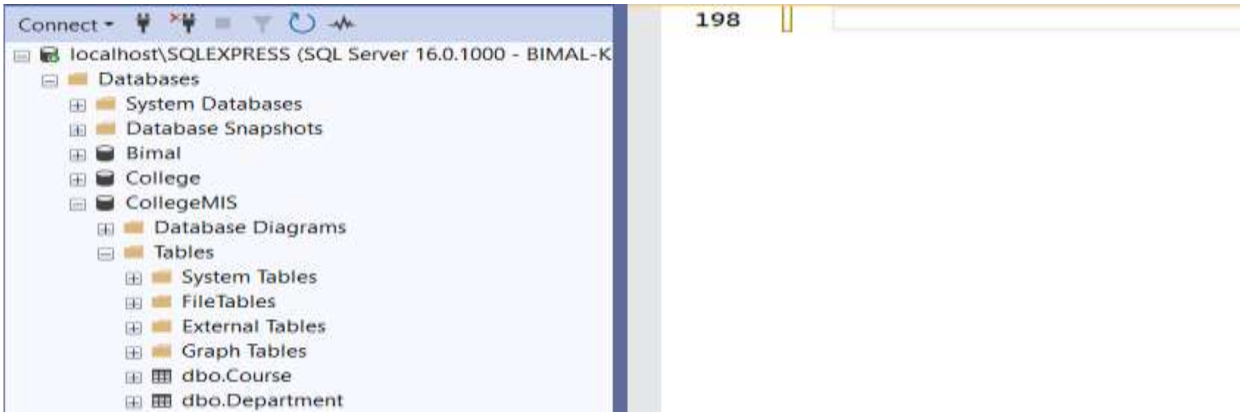
FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)

);



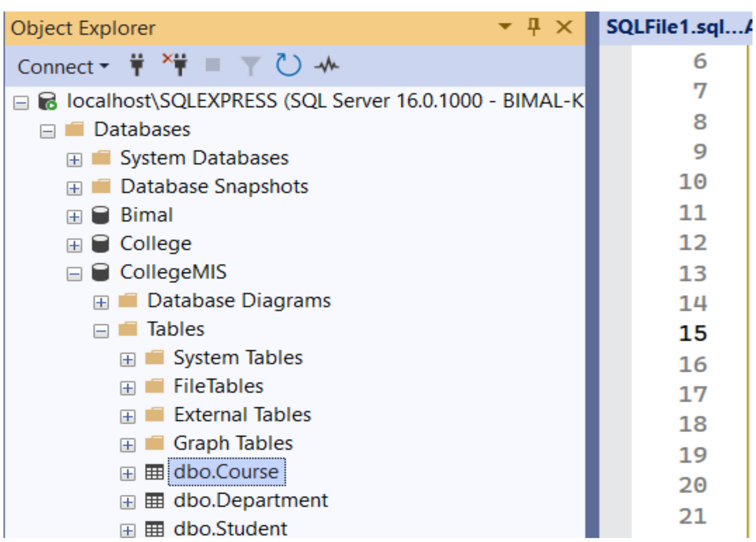
C. Creating Table of Department:

Query:
```
CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);
```



D. Creating Table of Course:

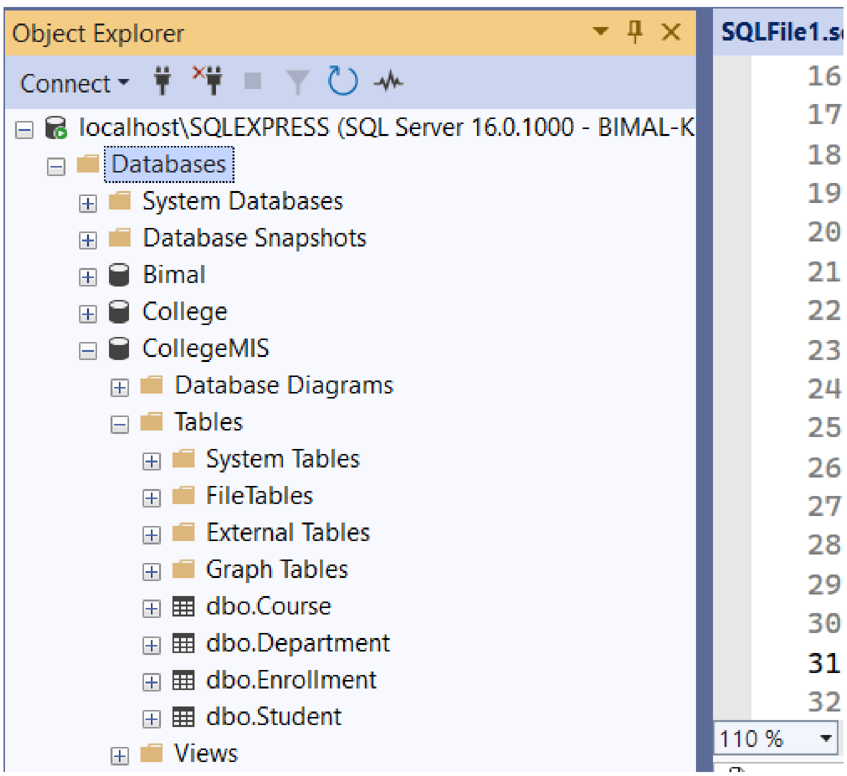
Query:
```
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100),
    DepartmentID INT,
        FOREIGN KEY (DepartmentID) REFERENCES
    Department(DepartmentID) );
```



E. Creating Table of Enrollement:

Query:

```
CREATE TABLE Enrollment (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    Semester VARCHAR(10),
    Marks INT,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```



1. Insert sample data into all four tables (Student, Department, Course, Enrollment).
   Query:

INSERT INTO StudentVALUES

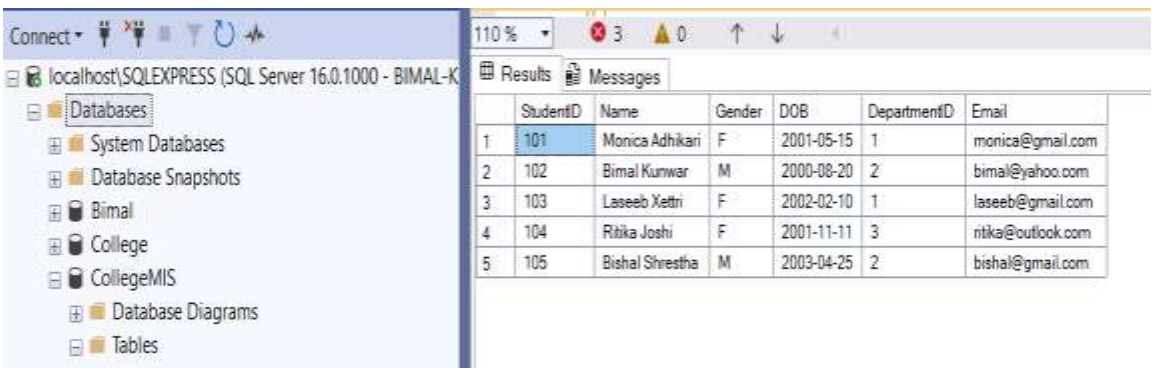
(101, 'Monica Adhikari', 'F', '2001-05-15', 1, 'monica@gmail.com'),

(102, 'Bimal Kunwar', 'M', '2000-08-20', 2, 'bimal@yahoo.com'),

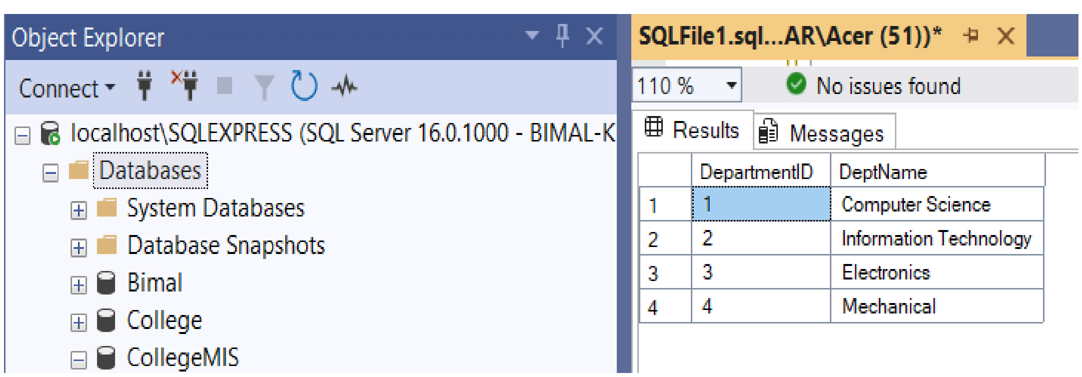
(103, 'Laseeb Xettri', 'F', '2002-02-10', 1, 'laseeb@gmail.com'),

(104, 'Ritika Joshi', 'F', '2001-11-11', 3, 'ritika@outlook.com'),

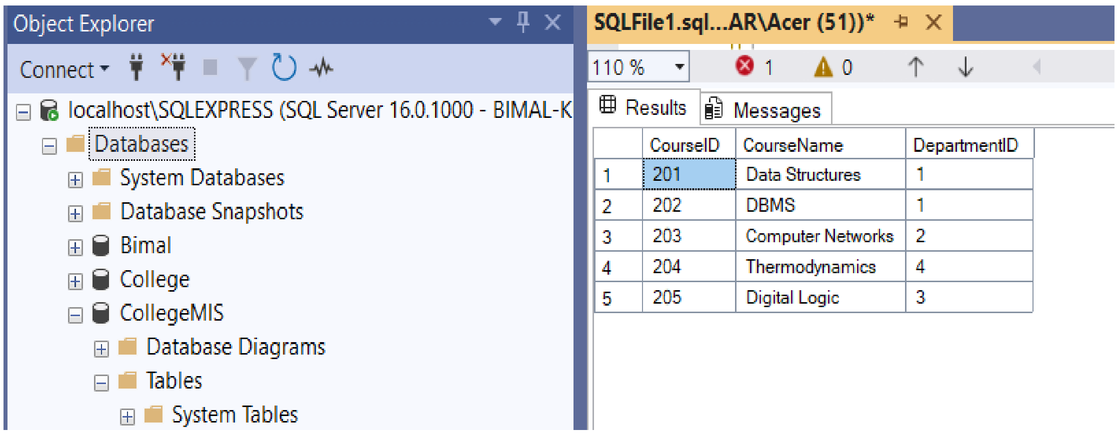
(105, 'Bishal Shrestha', 'M', '2003-04-25', 2, 'bishal@gmail.com');



INSERT INTO Department (DepartmentID, DeptName) VALUES
(1, 'Computer Science'),
(2, 'Information Technology'),
(3, 'Electronics'),
(4, 'Mechanical');

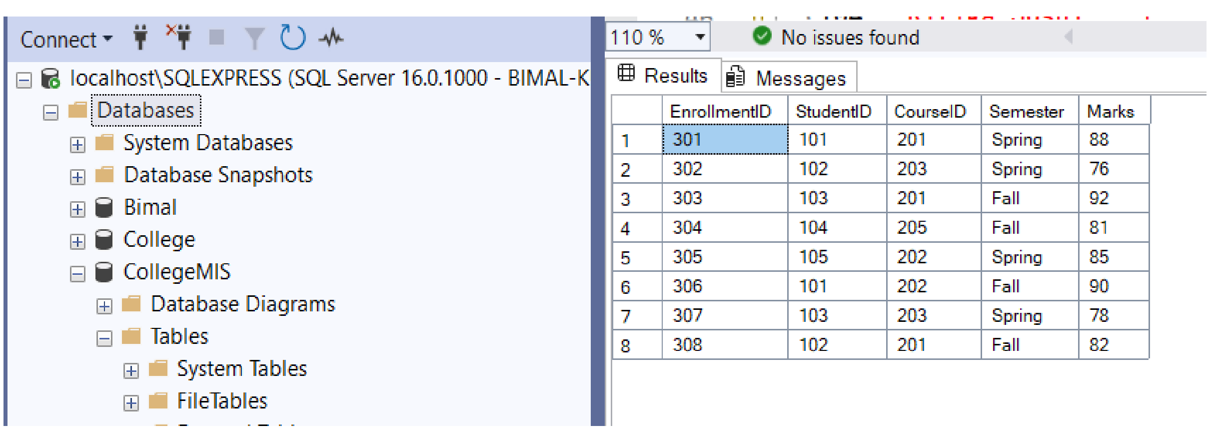


INSERT INTO Course VALUES
(201, 'Data Structures', 1),
(202, 'DBMS', 1),
(203, 'Computer Networks', 2),
(204, 'Thermodynamics', 4),
(205, 'Digital Logic', 3);



	CourseID	CourseName	DepartmentID
1	201	Data Structures	1
2	202	DBMS	1
3	203	Computer Networks	2
4	204	Thermodynamics	4
5	205	Digital Logic	3

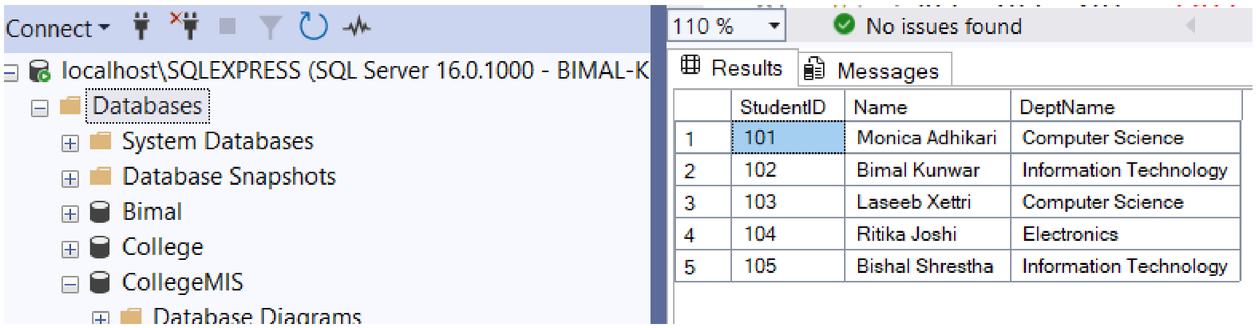
INSERT INTO Enrollment VALUES
(301, 101, 201, 'Spring', 88),
(302, 102, 203, 'Spring', 76),
(303, 103, 201, 'Fall', 92),
(304, 104, 205, 'Fall', 81),
(305, 105, 202, 'Spring', 85),
(306, 101, 202, 'Fall', 90),
(307, 103, 203, 'Spring', 78),
(308, 102, 201, 'Fall', 82);



	EnrollmentID	StudentID	CourseID	Semester	Marks
1	301	101	201	Spring	88
2	302	102	203	Spring	76
3	303	103	201	Fall	92
4	304	104	205	Fall	81
5	305	105	202	Spring	85
6	306	101	202	Fall	90
7	307	103	203	Spring	78
8	308	102	201	Fall	82

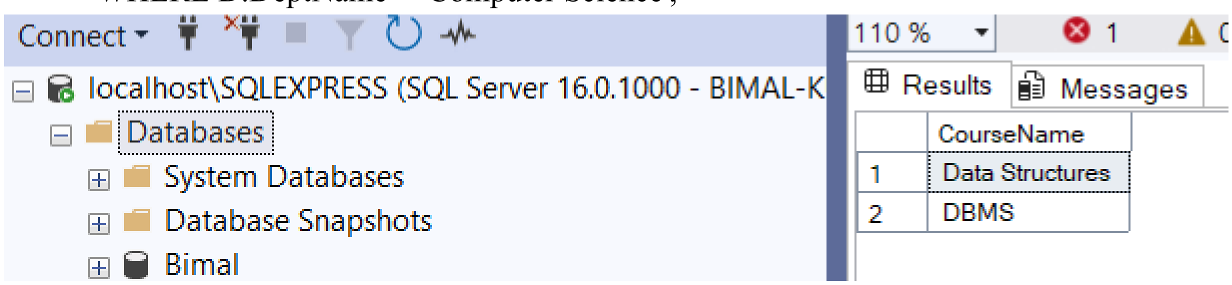
2. Display all students with their department name using JOIN.

Query: SELECT S.StudentID, S.Name, D.DeptName
FROM Student S
JOIN Department D ON S.DepartmentID = D.DepartmentID;



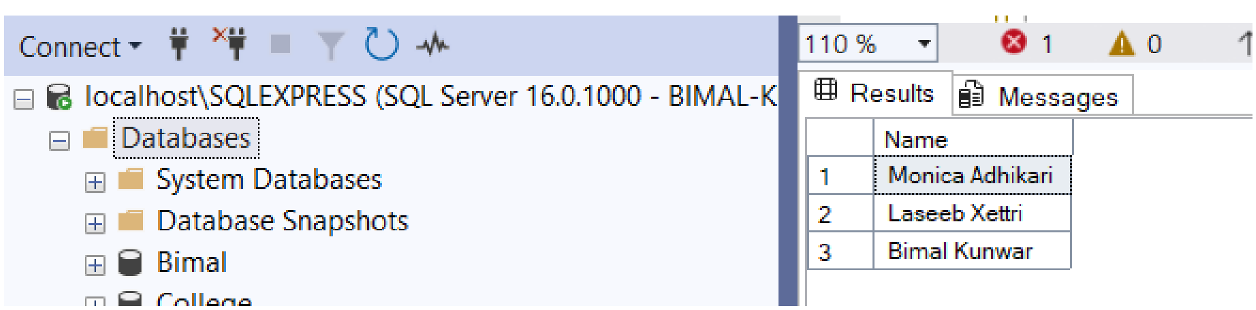
3. Show all courses offered by the "Computer Science" department.

Query: SELECT CourseName
FROM Course C
JOIN Department D ON C.DepartmentID = D.DepartmentID
WHERE D.DeptName = 'Computer Science';



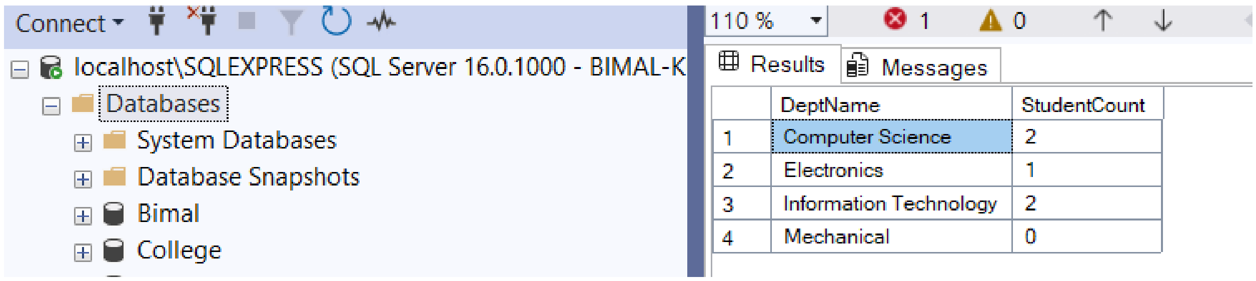
4. List all students enrolled in "Data Structures" course.

Query: SELECT DISTINCT S.Name
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID
WHERE C.CourseName = 'Data Structures';



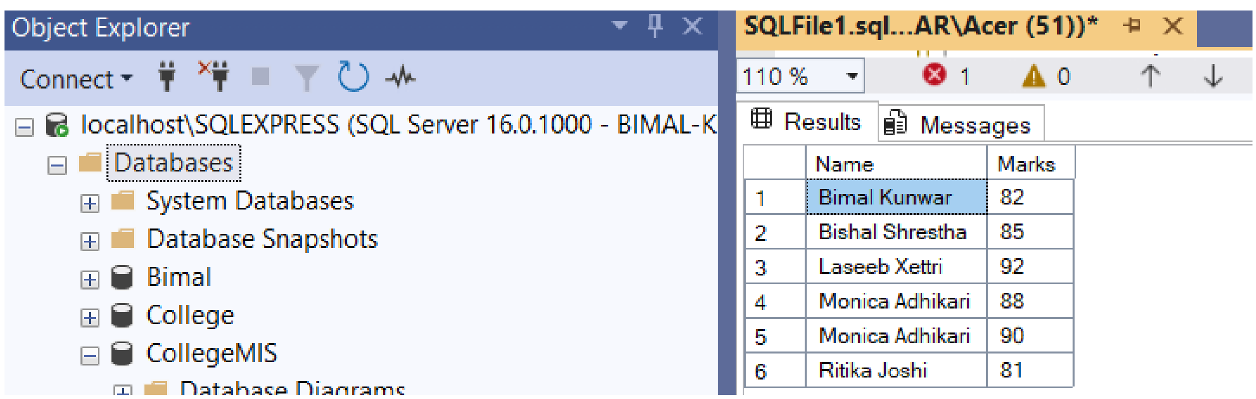
5. Count the number of students in each department.

Query: SELECT D.DeptName, COUNT(S.StudentID) AS StudentCount
FROM Department D
LEFT JOIN Student S ON D.DepartmentID = S.DepartmentID
GROUP BY D.DeptName;



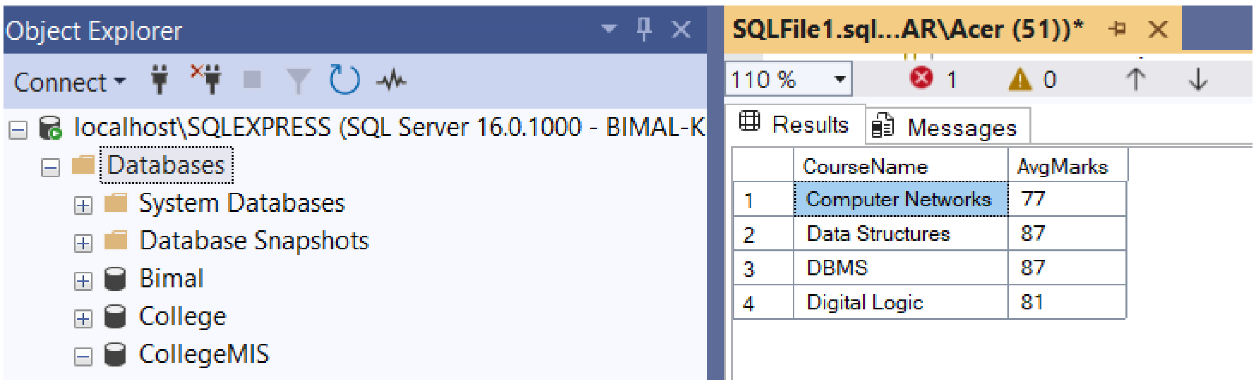
6. List students who scored more than 80 in any course.

Query: SELECT DISTINCT S.Name, E.Marks
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
WHERE E.Marks > 80;



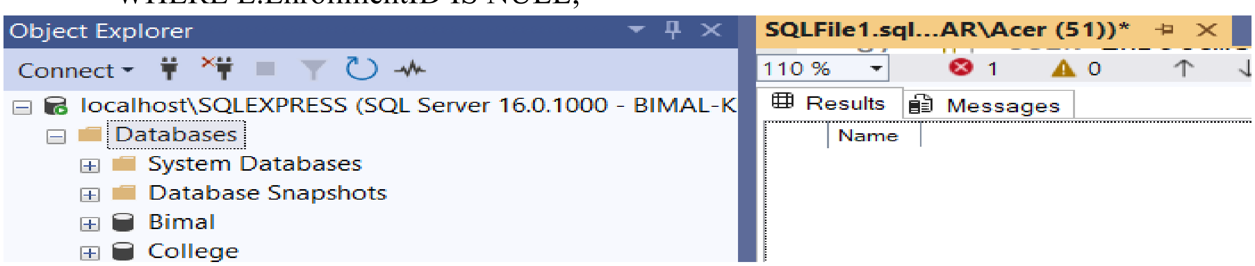
7. Find the average marks for each course using GROUP BY.

Query: SELECT C.CourseName, AVG(E.Marks) AS AvgMarks
FROM Course C
JOIN Enrollment E ON C.CourseID = E.CourseID
GROUP BY C.CourseName;



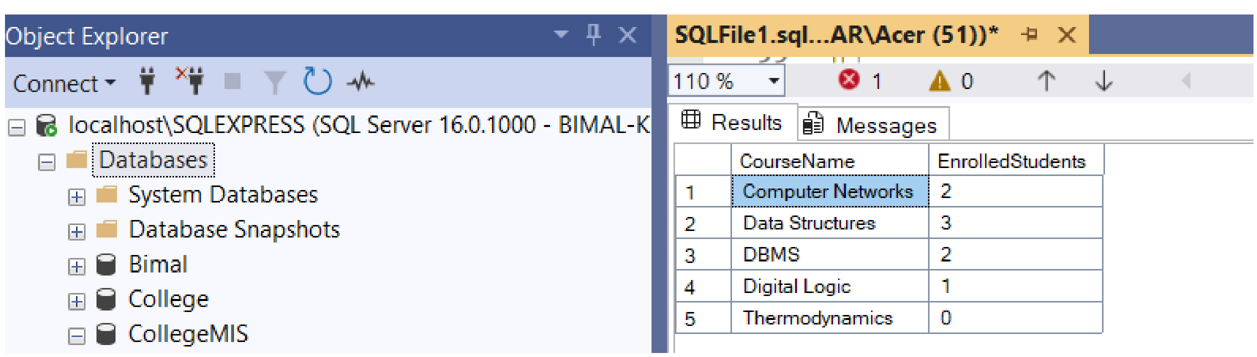
8. Display students who have not enrolled in any course.

Query: SELECT S.Name
FROM Student S
LEFT JOIN Enrollment E ON S.StudentID = E.StudentID
WHERE E.EnrollmentID IS NULL;



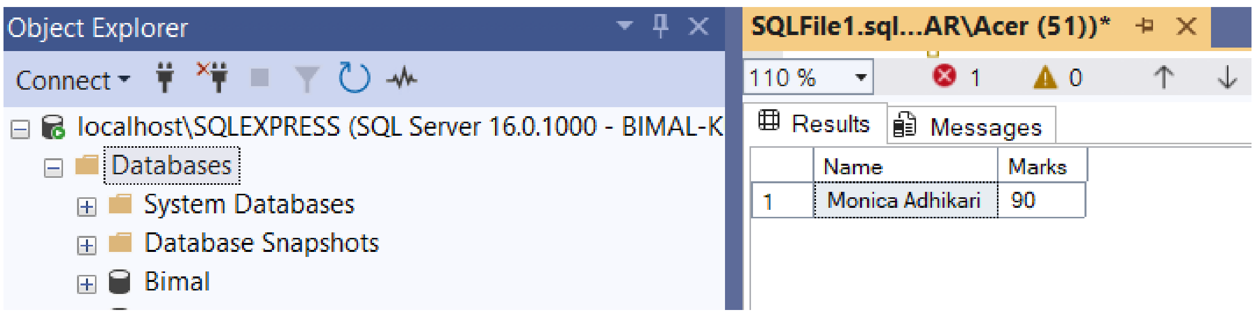
9. List all courses with the number of students enrolled in them.

Query: SELECT C.CourseName, COUNT(E.StudentID) AS EnrolledStudents
FROM Course C
LEFT JOIN Enrollment E ON C.CourseID = E.CourseID
GROUP BY C.CourseName;



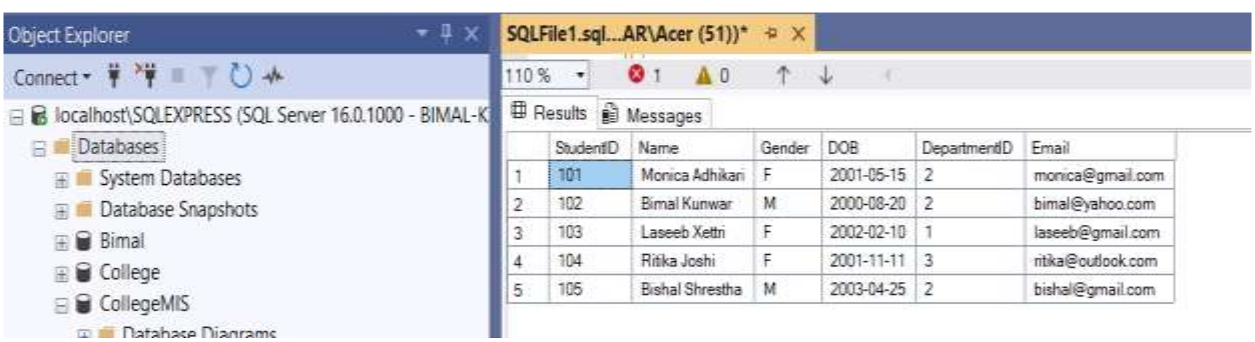
10. Find the student with the highest marks in "DBMS" course.

Query:
```
SELECT S.StudentID, S.Name, E.Marks
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID
WHERE C.CourseName = 'DBMS'
AND E.Marks = (
  SELECT MAX(E2.Marks)
 FROM Enrollment E2
JOIN Course C2 ON E2.CourseID = C2.CourseID
 WHERE C2.CourseName = 'DBMS'
);
```



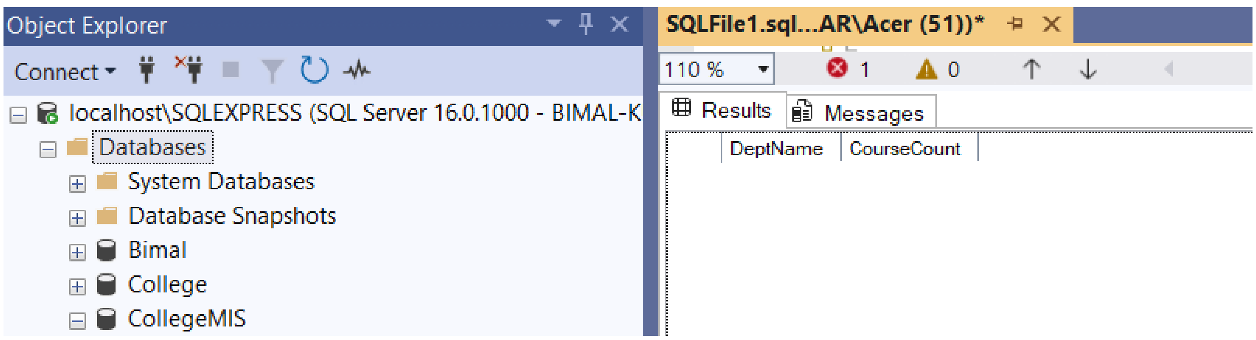
11. Update a student's department based on StudentID.

Query:
```
UPDATE Student
SET DepartmentID = 2
WHERE StudentID = 101;
```



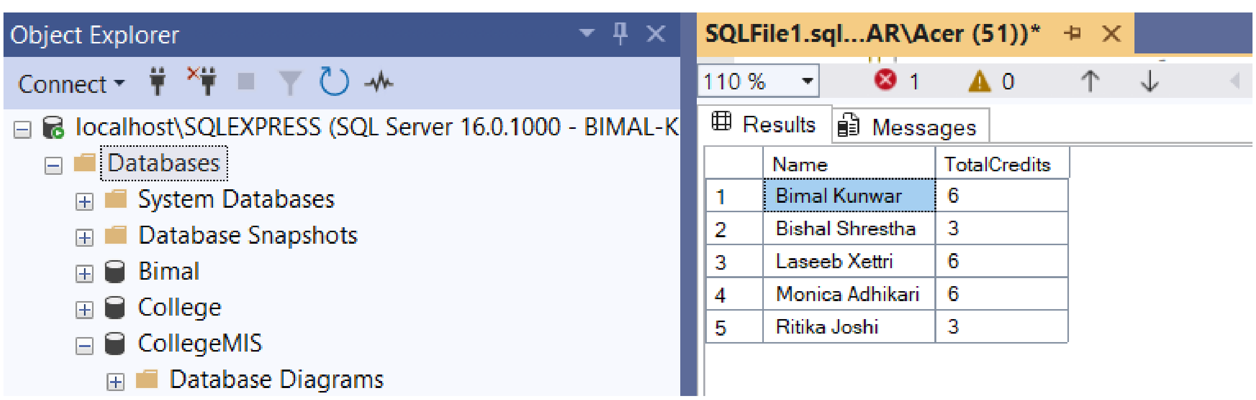
12. Find the departments that offer more than 3 courses.

Query: SELECT D.DeptName, COUNT(C.CourseID) AS CourseCount
FROM Department D
JOIN Course C ON D.DepartmentID = C.DepartmentID
GROUP BY D.DeptName
HAVING COUNT(C.CourseID) > 3;



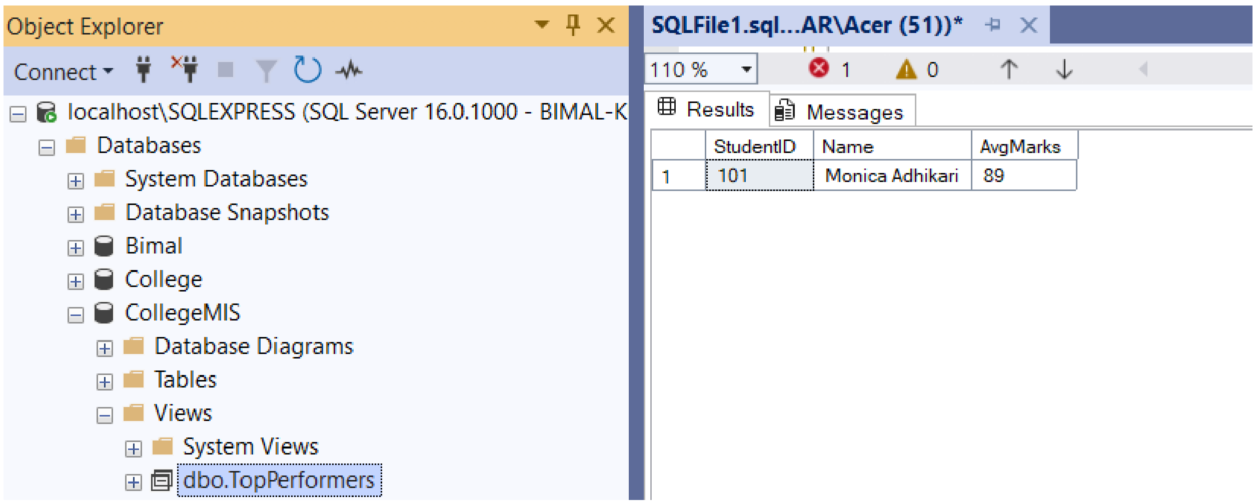
13. Show the names of students along with total credits they are enrolled in.

Query: SELECT S.Name, COUNT(E.CourseID) AS TotalCourses
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
GROUP BY S.Name;



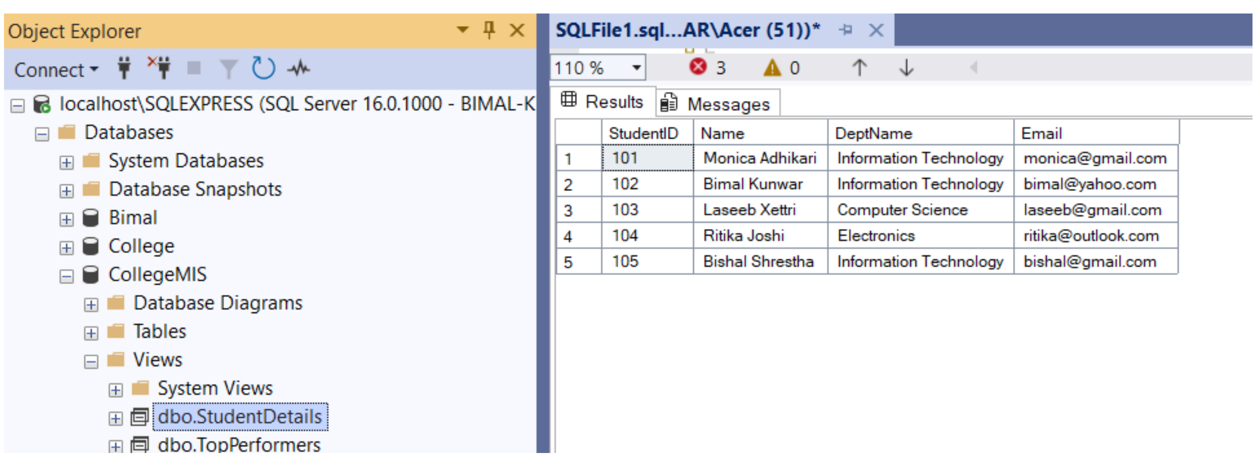
14. Create a view TopPerformers that shows students with average marks > 85.

Query: CREATE VIEW TopPerformers AS
SELECT S.StudentID, S.Name, AVG(E.Marks) AS AvgMarks
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
GROUP BY S.StudentID, S.Name
HAVING AVG(E.Marks) > 85;



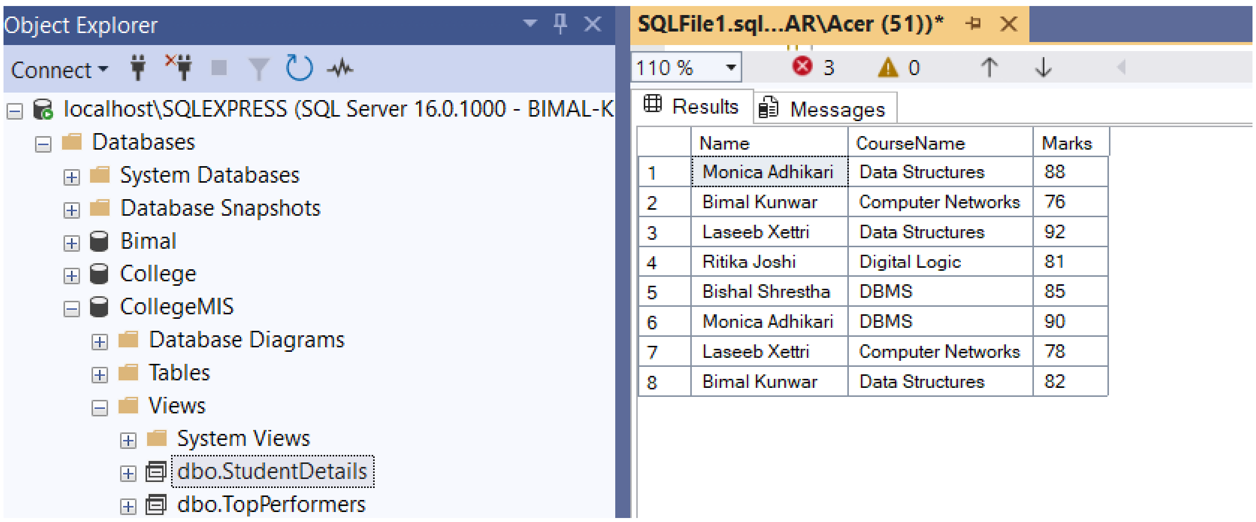
15. Create a view StudentDetails to show StudentID, Name, Department, and Email

Query: CREATE VIEW StudentDetails AS
SELECT S.StudentID, S.Name, D.DeptName AS Department, S.Email
FROM Student S
JOIN Department D ON S.DepartmentID = D.DepartmentID;



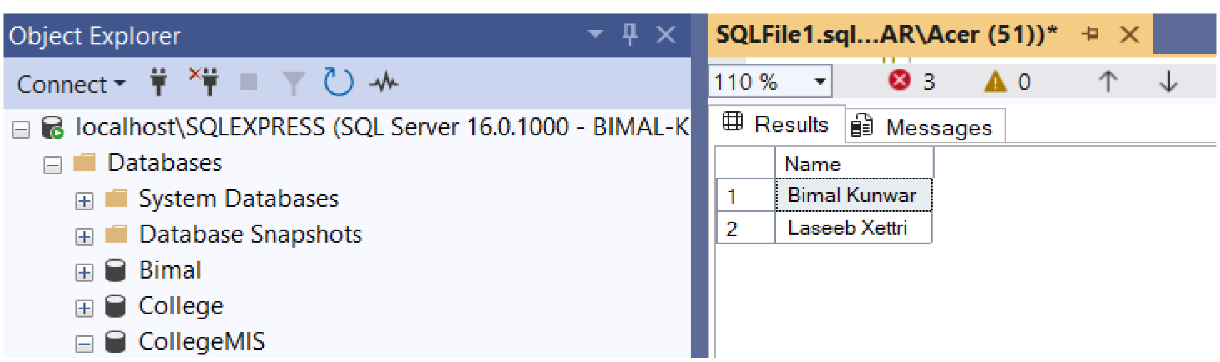
16. Write a query to show student name, course name, and marks for all enrollments

Query:
```
SELECT S.Name AS StudentName, C.CourseName, E.Marks
FROM Enrollment E
JOIN Student S ON E.StudentID = S.StudentID
JOIN Course C ON E.CourseID = C.CourseID;
```



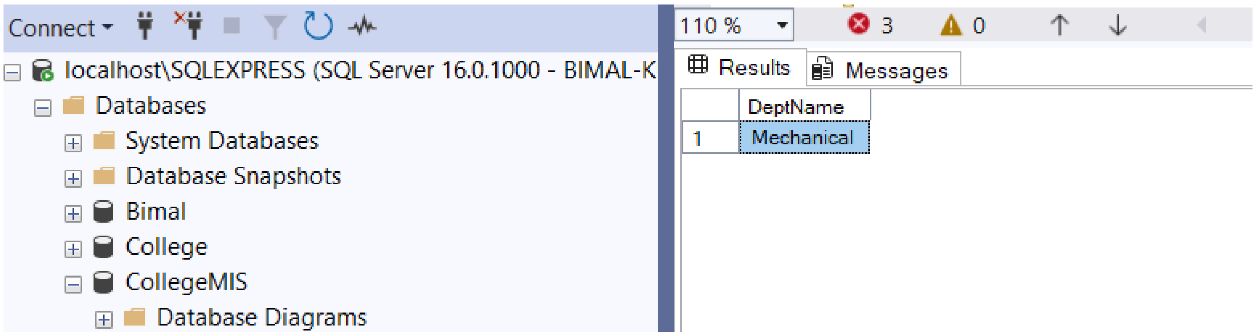
17. Find students who have enrolled in courses from more than one department.

Query:
```
SELECT S.Name
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID
GROUP BY S.StudentID, S.Name
HAVING COUNT(DISTINCT C.DepartmentID) > 1;
```



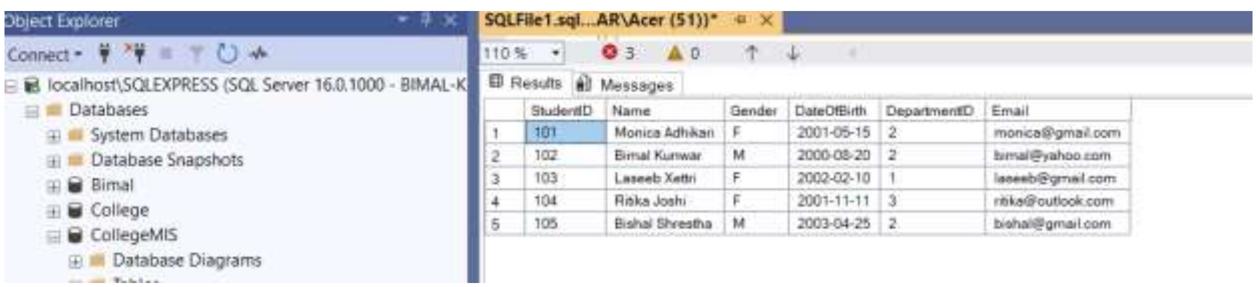
18. Display the list of departments that currently have no students

Query:
```
SELECT DeptName
FROM Department D
LEFT JOIN Student S ON D.DepartmentID = S.DepartmentID
WHERE S.StudentID IS NULL;
```



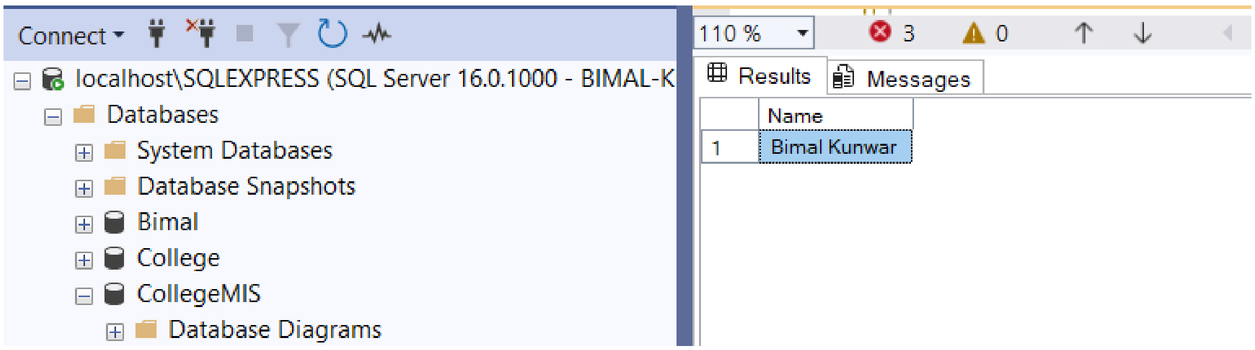
19. Rename the column DOB in Student to DateOfBirth.

Query:
```
EXEC sp_rename 'Student.DOB', 'DateOfBirth', 'COLUMN';
```



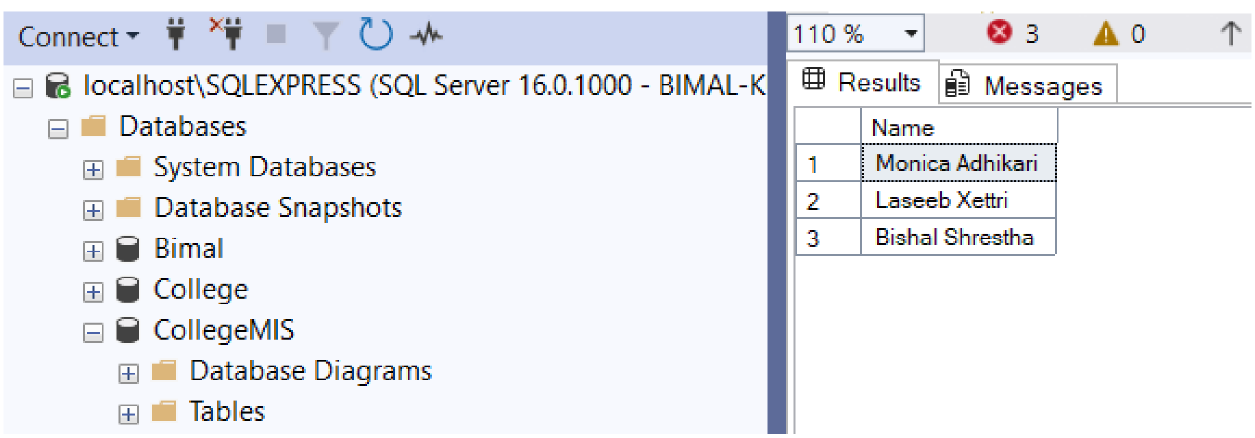
20. Find students who are both in the "IT" department and enrolled in "Computer Networks"

Query: SELECT S.Name
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID
JOIN Department D ON S.DepartmentID = D.DepartmentID
WHERE D.DeptName = 'IT' AND C.CourseName = 'Computer Networks';



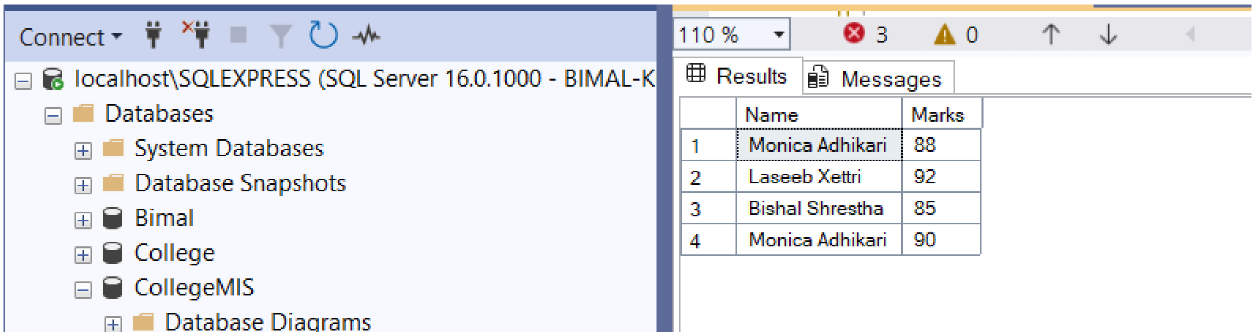
21. Find students whose email ends with '@gmail.com'

Query: SELECT Name, Email
FROM Student
WHERE Email LIKE '%@gmail.com';



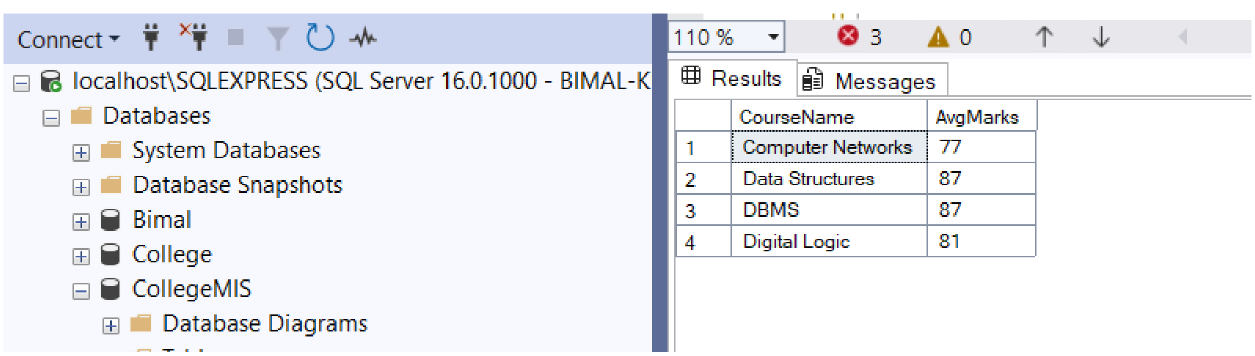
22. Display students who scored more than the average mark of all students.

Query:
```
SELECT S.Name, E.Marks
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
WHERE E.Marks > (SELECT AVG(Marks) FROM Enrollment);
```



23. Show the average marks of students in each course.

Query:
```
SELECT C.CourseName, AVG(E.Marks) AS AvgMarks
FROM Course C
JOIN Enrollment E ON C.CourseID = E.CourseID
GROUP BY C.CourseName;
```



24. Show all students with course name and sort by Marks descending.

Query:
```sql
SELECT S.Name AS StudentName, C.CourseName, E.Marks
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID
ORDER BY E.Marks DESC;
```